



FOUR-COURSE SYSTEM

COURSE 3 - DATA ENGINEERING 2026

C3-00 Engineering Foundations & Environment

Beginner-first canonical lesson book | 12 lessons | engineering habits before heavy tools

Artifact	C3_010_ENGINEERING_FOUNDATIONS_BEGINNER_FIRST_LESSON_BOOK_RC1
Status	2026 - LEARNER EDITION
Teaching standard	M01 beginner-first teacher loop; book remains sufficient without video.
Bridge boundary	Course 2B competencies may be assumed; all new engineering concepts still start from first principles.
Brand	Charcoal #1F2937 Teal #14B8A6 Soft Blue #3B82F6 AMOLED #000

House rule: understand -> follow once -> see expected result -> break/fix -> do a changed version -> validate -> explain -> save evidence. A running command alone never counts as mastery.

This book is your teacher

Use this book as the primary teacher. External videos and official documentation are visual or current-behavior companions, never hidden prerequisites.

The locked beginner-first lesson contract

- | | |
|-----------------------------------|---|
| 01. Why it matters | 09. Why it worked |
| 02. What you learn | 10. Common beginner mistakes |
| 03. Prerequisite / starting state | 11. If stuck / recovery |
| 04. Picture it in your head | 12. Small independent task |
| 05. New words before use | 13. Validate |
| 06. Useful video/source | 14. Teach it back |
| 07. Follow with me once | 15. Protected review + changed retry |
| 08. Expected result | 16. Evidence/mastery + exact next route |

Rules that protect your learning

- Save a genuine attempt before opening a protected review.
- After review, close it and solve a changed retry - do not copy the reviewed answer.
- A command that runs is not mastery: validate grain, keys, counts, state, failure/recovery and evidence.
- If a new engineering concept appears, this book explains it from first principles even when Bridge skills are assumed.
- When stuck, repair only the named weak competency; do not restart the whole phase.

Contents / Index

Actual page numbers after the master-audit repair. Mentor routes use these same pages.

Front matter	Page
This book is your teacher	2
Contents / Index	3
Module map + practice/review/Gate route	4
Phase-specific study notes	5
Lessons	Start
C3-00-L01 Role shift: analyst scripts -> engineering systems	6
C3-00-L02 Linux/WSL2 shell, paths and filesystem	9
C3-00-L03 Shell pipes, redirection and process basics	12
C3-00-L04 Git branches, commits and pull-request workflow	15
C3-00-L05 Python virtual environments and dependency pinning	18
C3-00-L06 Python modules, packages and project structure	21
C3-00-L07 Configuration, environment variables and secret boundaries	24
C3-00-L08 Structured logging and exception handling	27
C3-00-L09 Unit tests and integration-test boundaries	30
C3-00-L10 CLI design and reusable data utilities	33
C3-00-L11 Docker and container mental model	36
C3-00-L12 Idempotence, reproducibility and runbooks	39
Mastery checklist + revision quick reference	43

Module map + protected learning route

Engineering Foundations & Environment

- 01 Prerequisite / prior-phase Gate
- 02 Lesson mental model + vocabulary
- 03 Follow guided example once
- 04 Matching independent practice
- 05 Save genuine attempt
- 06 Protected review unlocks
- 07 Close review + changed retry
- 08 Validate + explain back
- 09 Master lesson + save evidence
- 10 Mini-Lab / integrated drill
- 11 Protected Gate A
- 12 If needed: protected review -> targeted repair -> changed micro-proof
- 13 Explicitly assign fresh Gate B/C only after repair evidence
- 14 Phase PASS -> next phase

What counts as mastery

- You can rebuild the competency without the lesson/review open.
- Your validation catches a plausible wrong result, not only syntax failure.
- You can state one failure mode and the recovery step.
- Your evidence names the input/output or artifact and the independent check used.

Revision rule

Revise from evidence: weak validation, weak recovery or weak explanation sends you back only to the matching lesson/repair route. Completion percentage never overrides a failed Gate.



How to study every lesson

This book teaches the concept before it asks you to perform independently. Videos are visual companions, never hidden prerequisites. Do not open the protected review until a genuine first attempt is saved.

Standard route: Why it matters -> What you learn -> Prerequisite -> Picture it -> New words -> Useful visual/source -> Tiny worked example -> Follow with me once -> Expected result -> Why it worked -> Break/fix -> Independent changed version -> Validate -> Teach it back -> Protected review -> Changed retry -> Mastery evidence.

Tool-install rule: install a tool only when its first-use lesson reaches it. WSL first use = L02; Git = L04; Python/venv = L05; pytest = L05/L09; Docker = L11. Heavy C3 tools such as Airflow, Spark and Kafka are not installed in C3-00.

Bridge rule: Course 2B already built analytical SQL, Python automation, modeling, warehousing/data quality and cloud awareness. C3-00 deliberately progresses those habits into engineering operation; it does not reteach the Bridge.

C3-00-L01 - Role shift: analyst scripts -> engineering systems

Focused effort: 2.0 h | Matching practice: P01 | Review stays protected until first attempt is saved.

WHY IT MATTERS

A one-off script can give the right answer once and still be unsafe to operate. Data engineering starts when another person can run, trust, diagnose, rerun and hand off the work without relying on your memory.

WHAT YOU LEARN

Turn a tiny data script into an explicit engineering contract covering input, output, grain, failure, rerun, validation, ownership and evidence.

PREREQUISITE

C2B capstone-level analytical ownership. You do not need orchestration, Docker, Spark or cloud experience yet.

PICTURE IT IN YOUR HEAD

Picture two kitchens. In the first, one cook remembers every step. In the second, ingredients are labeled, measurements are written, failed batches are rejected, and another cook can reproduce the result. Engineering is the second kitchen.

NEW WORDS BEFORE WE USE THEM

Word	Meaning in this lesson
contract	A testable statement of what the system accepts, produces and guarantees.
grain	What one row or record represents.
idempotent	Safe to rerun without accidentally multiplying or corrupting the intended result.
observable	A failure or important state leaves evidence you can inspect.
runbook	Short operating instructions for run, validation, failure and recovery.

USEFUL VIDEO / SOURCE

REQUIRED REFERENCE: Course-built engineering contract

Purpose: Engineering-system mental model and contract checklist

Useful segment: Entire lesson; no external dependency

PLAIN-ENGLISH EXPLANATION

Correctness is necessary but not sufficient. A production-style utility needs an input contract, stable command, known output grain, visible failure, independent checks, a rerun rule, logs/evidence, and an owner/handoff path. The contract is written before implementation so you know what "done" means.

C3-00-L01 - Follow with me once

TINY WORKED EXAMPLE

Input sales.csv has one row per order. Business rule: include only status=Completed. Output summary.json has one row-equivalent object with completed_order_count and net_revenue. If required columns are missing, exit non-zero; never silently report 0 revenue.

FOLLOW WITH ME ONCE

1. Create `P01\_contract.md`.
2. Write INPUT, INPUT GRAIN, REQUIRED COLUMNS, BUSINESS RULE, OUTPUT, OUTPUT GRAIN, FAILURE, RERUN, VALIDATION, OWNER, EVIDENCE.
3. Add three failure scenarios: missing required column, unreadable file, and duplicate rerun.
4. For every acceptance criterion, ask: could a reviewer objectively test this?

COMMAND / STRUCTURE TO TYPE

```
INPUT: sales.csv | grain=one row/order | required=order_id,status,qty,unit_price
OUTPUT: out/summary.json | grain=one summary per run
FAIL: missing column -> exit 2 + error log; do not publish output
RERUN: same input/config -> same business result, no duplicated rows
VALIDATE: source Completed count == output completed_order_count
```

EXPECTED RESULT

Your contract is specific enough that two people would build the same behavior. "Looks right" does not appear as a validation rule.

WHY IT WORKED

The rule separates business meaning from implementation. Even before code exists, expected behavior, failure and validation are inspectable.

BREAK / FIX / DEBUG

Break it: remove `status` from the input. Bad design prints revenue 0 and exits 0. **Fix:** validate schema before calculation, emit a clear error, exit non-zero, and leave the previous good output untouched.

COMMON BEGINNER MISTAKES

- Starting code before stating input/output grain.
- Treating a plausible number as evidence.
- Writing "rerun script" without saying whether overwrite/replay is safe.
- Making the original developer the only person who knows what to check.

RECOVERY IF STUCK

If the contract feels abstract, write one valid row, one invalid row, one expected output, and one rerun story. Turn those concrete examples into rules.



C3-00-L01 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

New domain: support tickets. Write a fresh contract for a utility that creates daily closed-ticket counts by queue. Do not copy the sales wording.

VALIDATION - DO NOT SKIP

- Every requirement can be tested yes/no.
- Input and output grain are explicit.
- At least one independent control would catch a plausible wrong result.
- Failure and rerun behavior are explicit.

PRODUCTION / ARCHITECTURE REASONING

Later Airflow/Spark/Fabric work uses the same contract: inputs, outputs, dependencies, failure, replay, validation, ownership. Tools change; this reasoning persists.

TEACH IT BACK

In 60-90 seconds explain role shift: analyst scripts -> engineering systems in simple English, including one failure mode and how you validate correctness.

PROTECTED REVIEW + CHANGED RETRY

1) Save genuine P01 attempt in an append-only attempt folder. 2) Only then open the matching P01 section of C3-012. 3) Record the mistake/reason, close the review, and do the changed retry without copying the example. 4) If still weak, use targeted remediation; do not restart the whole phase.

EVIDENCE / MASTERY

Save P01 first attempt, three failure scenarios, changed support-ticket contract, one independent validation rule, and a 60-90 second explain-back.

Mastered only when: the changed retry passes its validation, you can explain the concept without the lesson/review open, and the evidence bundle is saved. A command merely running is not mastery.

C3-00-L02 - Linux/WSL2 shell, paths and filesystem

Focused effort: 2.5 h | Matching practice: P02 | Review stays protected until first attempt is saved.

WHY IT MATTERS

Most data-engineering tools assume Linux paths and shell behavior. Path confusion can look like a Python, Git or Docker failure when the real problem is simply "you are in the wrong place."

WHAT YOU LEARN

Install/verify WSL2 if needed, navigate a Linux-native project safely, distinguish absolute/relative paths, and recover from path confusion without destructive commands.

PREREQUISITE

Windows basics only. WSL is installed now only if it is missing.

PICTURE IT IN YOUR HEAD

Think of Windows and WSL as two connected buildings. ``mnt/c`` is a bridge to the Windows building; ``~/course3/...`` is your desk inside the Linux building. Linux-heavy work is usually smoother when the project lives at the Linux desk.

NEW WORDS BEFORE WE USE THEM

Word	Meaning in this lesson
shell	Text interface that starts programs and connects their input/output.
distribution	A Linux installation such as Ubuntu running under WSL.
absolute path	Full path starting at <code>`/`</code> .
relative path	Path interpreted from your current directory.
home <code>`~`</code>	Your Linux user home directory.
<code>/mnt/c</code>	WSL mount exposing the Windows C: drive.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [Microsoft - Install WSL](#)

Purpose: Install only if WSL is missing; verify WSL2

Useful segment: Use install + verify sections

RECOMMENDED REFERENCE: [Microsoft - Set up a WSL development environment](#)

Purpose: Linux-native project path and Windows/WSL boundary

Useful segment: File-storage guidance

RECOMMENDED VIDEO: [Microsoft - What is WSL? One Dev Question](#)

Purpose: Very short Microsoft visual orientation before first shell session

Useful segment: 00:00-end (short orientation)

PLAIN-ENGLISH EXPLANATION

``pwd`` tells you where you are; ``ls -lah`` shows what is there; ``cd`` moves; ``mkdir -p`` creates directories; ``cp`` copies; ``mv`` moves/renames. Inspection comes before deletion. Course projects live under ``~/course3`` unless a lesson explicitly says otherwise.



C3-00-L02 - Follow with me once

TINY WORKED EXAMPLE

Create `~/course3/c3_00/fs_lab/{input,output,archive,docs}`. Put `names.txt` in input, copy it to archive, move a note into docs, then print the final tree.

FOLLOW WITH ME ONCE

1. In Windows PowerShell run `wsl -l -v`. If no distribution exists, open Administrator PowerShell and run `wsl --install`, restart if requested, then create your Linux username.
2. Verify the distribution shows VERSION 2.
3. Inside WSL run `pwd` and `realpath ~`.
4. Create the C3-00 Linux-native workspace and save commands into `commands.txt`.
5. Use `explorer.exe .` only to inspect the Linux folder from Windows; keep the project itself in WSL.

COMMAND / STRUCTURE TO TYPE

```
# Windows PowerShell
wsl -l -v
# If missing (Administrator PowerShell):
wsl --install

# Inside WSL
mkdir -p ~/course3/c3_00/fs_lab/{input,output,archive,docs}
cd ~/course3/c3_00/fs_lab
pwd
printf "Ada\nRavi\n" > input/names.txt
cp input/names.txt archive/names_backup.txt
printf "filesystem lab\n" > note.txt
mv note.txt docs/README_NOTE.txt
find . -maxdepth 2 -type f -print
```

EXPECTED RESULT

`wsl -l -v` lists your Linux distribution with VERSION 2. Inside WSL, `pwd` for the course workspace begins with `/home/...`, not `/mnt/c/...`.

WHY IT WORKED

The commands create a predictable Linux-native layout, prove your current directory, and preserve the source while you practice copy/move operations.

BREAK / FIX / DEBUG

Break it: run a relative command from the wrong directory. **Fix sequence:** `pwd -> ls -lah -> realpath . -> inspect target -> then rerun`. Never add `sudo` or `rm -rf` as a generic fix.

COMMON BEGINNER MISTAKES

- Keeping the whole Linux-heavy project under `/mnt/c` because it looks familiar.
- Using `sudo` whenever permission/path reasoning is unclear.
- Running `rm -rf` before checking `pwd`.
- Confusing `~`, `..`, `...` and `/`.

RECOVERY IF STUCK

Use inspection-only commands first: `pwd`, `ls -lah`, `realpath .`, `find . -maxdepth 2 -type f`. If unsure, stop before move/delete.



C3-00-L02 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

Create `~/course3/c3_00/path_transfer/{incoming,processed,archive}` from memory. Copy one file to archive, move a different file to processed, then prove the source/target state.

VALIDATION - DO NOT SKIP

- Project path is Linux-native.
- Source file you intended to preserve still exists.
- Final tree matches your plan.
- You can explain every relative path from the directory where the command ran.

PRODUCTION / ARCHITECTURE REASONING

Airflow, Spark, Docker and CI/CD troubleshooting often begins with environment/path inspection. This habit prevents "fixes" that damage a correct environment.

TEACH IT BACK

In 60-90 seconds explain WSL, Linux-native path, `/mnt/c`, absolute vs relative path, and your safe recovery sequence when lost.

PROTECTED REVIEW + CHANGED RETRY

1) Save genuine P02 attempt in an append-only attempt folder. 2) Only then open the matching P02 section of C3-012. 3) Record the mistake/reason, close the review, and do the changed retry without copying the example. 4) If still weak, use targeted remediation; do not restart the whole phase.

EVIDENCE / MASTERY

Save `wsl -l -v`, `pwd`, final file tree, `commands.txt`, changed transfer, and a 60-90 second explanation of Linux-native vs `/mnt/c` paths.

Mastered only when: the changed retry passes its validation, you can explain the concept without the lesson/review open, and the evidence bundle is saved. A command merely running is not mastery.

C3-00-L03 - Shell pipes, redirection and process basics

Focused effort: 2.0 h | Matching practice: P03 | Review stays protected until first attempt is saved.

WHY IT MATTERS

Pipelines are chains of small programs. Shell pipes teach the same mental model in miniature: one stage produces output, the next consumes it, and failures must be visible.

WHAT YOU LEARN

Use stdin/stdout/stderr, pipes, redirection and exit status safely; debug a command chain one stage at a time.

PREREQUISITE

L02 filesystem/navigation.

PICTURE IT IN YOUR HEAD

Imagine a row of buckets. stdout pours into the next bucket through ``|``. stderr is a separate red warning pipe. ``>`` replaces a destination bucket; ``>>`` adds to it. The exit code is the stage signal: success or a specific kind of non-success.

NEW WORDS BEFORE WE USE THEM

Word	Meaning in this lesson
stdin	Input stream a program can read.
stdout	Normal output stream.
stderr	Error/diagnostic output stream.
pipe <code>` `</code>	Connect stdout of the left command to stdin of the right.
redirection	Send a stream to/from a file.
exit status	Integer result code; 0 conventionally means success.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [GNU Bash Reference Manual](#)

Purpose: Pipes, redirection, exit status

Useful segment: Redirections + pipelines + exit status

PLAIN-ENGLISH EXPLANATION

A pretty printed message is not the same as success. Automation must know the exit status. Also, some commands use non-zero for a meaningful no-match state rather than a crash, so interpret the command contract rather than assuming every non-zero means the same thing.

C3-00-L03 - Follow with me once

TINY WORKED EXAMPLE

Count repeated log levels, save warnings, and intentionally search for a label that is absent. Inspect ` \$? ` immediately after the search.

FOLLOW WITH ME ONCE

1. Create `events.log` with INFO/WARN/ERROR lines.
2. Run `cut` or `awk` to extract the first field, then `sort | uniq -c`.
3. Use `grep WARN events.log > warnings.txt`.
4. Run `grep FATAL events.log`; immediately run `echo \$?`.
5. Break the chain into single commands if any stage surprises you.

COMMAND / STRUCTURE TO TYPE

```
printf 'INFO start\nWARN slow\nINFO done\nERROR bad\nWARN retry\n' > events.log
awk '{print $1}' events.log | sort | uniq -c > level_counts.txt
grep WARN events.log > warnings.txt
grep FATAL events.log
echo $?
```

EXPECTED RESULT

The level counts are deterministic, `warnings.txt` contains only WARN lines, the source log is unchanged, and the intentional no-match exit code is recorded.

WHY IT WORKED

Each stage has one responsibility and leaves inspectable evidence. Redirection creates artifacts without editing the source; immediate exit-status inspection prevents guessing.

BREAK / FIX / DEBUG

Break it: pipe three commands and get an empty output. **Fix:** run stage 1 alone, then stage 1|2, inspect, then add stage 3. Also check filename and current directory before changing permissions.

COMMON BEGINNER MISTAKES

- Using `>` when you meant `>>` and overwriting evidence.
- Checking ` \$? ` after another command, which replaces the status you wanted.
- Assuming empty output means the command crashed.
- Building a long pipeline before validating each stage.

RECOVERY IF STUCK

Start with the smallest stage, save intermediate output, inspect stderr, and record exit codes. Reassemble only after each stage behaves as expected.



C3-00-L03 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

On a changed `service.log`, produce counts by level and a separate `ERROR` artifact without modifying the input. Add one deliberate no-match check and explain its result.

VALIDATION - DO NOT SKIP

- Source hash is unchanged.
- Counts reconcile to total lines.
- Filtered artifact contains only the requested label.
- Exit status is captured immediately and interpreted correctly.

PRODUCTION / ARCHITECTURE REASONING

Orchestrators and containers still run processes with streams and exit codes. A task that silently exits 0 after bad data is a production reliability bug.

TEACH IT BACK

In 60-90 seconds explain shell pipes, redirection and process basics in simple English, including one failure mode and how you validate correctness.

PROTECTED REVIEW + CHANGED RETRY

1) Save genuine P03 attempt in an append-only attempt folder. 2) Only then open the matching P03 section of C3-012. 3) Record the mistake/reason, close the review, and do the changed retry without copying the example. 4) If still weak, use targeted remediation; do not restart the whole phase.

EVIDENCE / MASTERY

Save shell commands, output files, source/output row controls, one intentional non-zero case, changed transfer, and explain-back.

Mastered only when: the changed retry passes its validation, you can explain the concept without the lesson/review open, and the evidence bundle is saved. A command merely running is not mastery.

C3-00-L04 - Git branches, commits and pull-request workflow

Focused effort: 2.5 h | Matching practice: P04 | Review stays protected until first attempt is saved.

WHY IT MATTERS

Engineering work must be reviewable and reversible. Git gives you inspectable history; branches isolate change; a pull request turns a diff into a review conversation.

WHAT YOU LEARN

Inspect working/staged/committed states, make a feature branch, create coherent commits, and prepare a PR-quality handoff without tracking secrets or disposable environments.

PREREQUISITE

L02-L03. Git is installed now only if ``git --version`` fails.

PICTURE IT IN YOUR HEAD

Think of three desks: **working tree** = what you are editing now; **staging area** = the exact pages you chose for the next snapshot; **commit history** = labeled snapshots already filed. A **branch** is a safe side lane; a **PR** asks someone to review merging that lane.

NEW WORDS BEFORE WE USE THEM

Word	Meaning in this lesson
repository	Project plus its Git history.
working tree	Current checked-out files.
staging area	Files/changes selected for the next commit.
commit	Named snapshot of staged changes.
branch	Movable line of development.
pull request	Review proposal to merge one branch into another.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [Git - git status](#)

Purpose: Working tree/staging inspection

Useful segment: Synopsis + short status behavior

RECOMMENDED REFERENCE: [Git - git switch](#)

Purpose: Create/switch branches safely

Useful segment: git switch -c

RECOMMENDED REFERENCE: [GitHub - Hello World](#)

Purpose: Branch/commit/pull-request flow

Useful segment: Branch -> commit -> pull request

RECOMMENDED VIDEO: [GitHub - How to create a pull request in 4 min](#)

Purpose: Visualize what a pull request is and how review fits the branch flow

Useful segment: 00:17-00:42 concept; 00:59-02:27 demo; 02:27-end tips

PLAIN-ENGLISH EXPLANATION

``git status`` is your first inspection tool. ``git diff`` shows unstaged change; ``git diff --staged`` shows what the next commit would contain. A good commit has one coherent purpose. Git is not a backup "by magic" unless history is safely stored/remotely shared.



C3-00-L04 - Follow with me once

TINY WORKED EXAMPLE

Create a feature branch, add a small validator in commit 1, add tests/docs in commit 2, then write a PR summary: what changed, why, how tested, risks/rollback.

FOLLOW WITH ME ONCE

1. Run `git --version`; if missing in WSL: `sudo apt update && sudo apt install -y git`, then verify.
2. Initialize a practice repository and create `.gitignore` containing `.venv/`, `.env`, `__pycache__`, `/*.pyc`.
3. Create `feature/add-validator` with `git switch -c`.
4. Before every commit: `git status`, `git diff`, `git add`, `git diff --staged`.
5. Create two focused commits and inspect `git log --oneline --decorate -5`.

COMMAND / STRUCTURE TO TYPE

```
git --version
git init
printf '.venv/\n.env\n__pycache__/\n*.pyc\n' > .gitignore
git add .gitignore README.md
git commit -m 'chore: initialize C3-00 practice project'
git switch -c feature/add-validator
# edit files
git status
git diff
git add src tests
git diff --staged
git commit -m 'feat: validate required input columns'
git log --oneline --decorate -5
```

EXPECTED RESULT

The feature branch has two understandable commits. `.env`, `.venv` and bytecode are not tracked. `git status` is clean after commits.

WHY IT WORKED

Inspection happens before mutation. The branch protects main, staging controls the exact snapshot, and commit messages describe intent rather than "changes".

BREAK / FIX / DEBUG

Break it: accidentally stage `.env`. Fix without deleting the local file: `git restore --staged .env`, add `.env` to `.gitignore`, inspect status, and confirm no secret entered commit history. If a real secret was committed, rotate it; do not rely on deletion alone.

COMMON BEGINNER MISTAKES

- `git add .` without reviewing the diff.
- One giant commit mixing code, generated outputs and experiments.
- Tracking `.venv`, `.env`, logs or bytecode.
- Using destructive `reset/clean` commands before understanding status.

RECOVERY IF STUCK

Run `git status`, then `git diff` and `git diff --staged`. Prefer reversible operations. Do not use `reset --hard` / `clean -fd` as reflexes.

C3-00-L04 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

Create `feature/logging-contract` in a fresh small repo, change one logger behavior, add/update one test, make two coherent commits, and draft a PR summary.

VALIDATION - DO NOT SKIP

- Branch name and commit history show the intended work.
- No secret/disposable environment is tracked.
- Diff is understood file by file.
- PR summary states what/why/how-tested/risk.

PRODUCTION / ARCHITECTURE REASONING

Git history plus PR evidence is the basis for CI/CD, deployment review, incident rollback and team handoff later in C3.

TEACH IT BACK

In 60-90 seconds explain git branches, commits and pull-request workflow in simple English, including one failure mode and how you validate correctness.

PROTECTED REVIEW + CHANGED RETRY

1) Save genuine P04 attempt in an append-only attempt folder. 2) Only then open the matching P04 section of C3-012. 3) Record the mistake/reason, close the review, and do the changed retry without copying the example. 4) If still weak, use targeted remediation; do not restart the whole phase.

EVIDENCE / MASTERY

Save branch/status/log output, clean secret check, PR-style summary, changed transfer, and explain branch vs commit vs PR.

Mastered only when: the changed retry passes its validation, you can explain the concept without the lesson/review open, and the evidence bundle is saved. A command merely running is not mastery.

C3-00-L05 - Python virtual environments and dependency pinning

Focused effort: 2.5 h | Matching practice: P05 | Review stays protected until first attempt is saved.

WHY IT MATTERS

"It works on my laptop" is not reproducibility. Your source and dependency instructions must be able to recreate a working environment after the original ``.venv`` is deleted.

WHAT YOU LEARN

Create/activate a venv, verify the active interpreter, install pytest, record dependencies, delete/recreate the environment, and prove tests still pass.

PREREQUISITE

L04. Python is installed now only if ``python3 --version`` fails.

PICTURE IT IN YOUR HEAD

A virtual environment is a temporary workshop rented for one project. You do not ship the whole workshop. You save the blueprint (source + dependency declaration) so another workshop can be rebuilt.

NEW WORDS BEFORE WE USE THEM

Word	Meaning in this lesson
interpreter	Program that executes Python code.
virtual environment	Project-isolated Python interpreter/package environment.
dependency	External package your project needs.
pin	Record a concrete version for reproducible installation.
recreate	Delete disposable environment and rebuild from durable files.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [Python 3.14 - venv](#)

Purpose: Disposable/recreatable virtual environments

Useful segment: Creation, activation, recreation

RECOMMENDED REFERENCE: [Python 3.14 - Installing modules](#)

Purpose: pip/dependency installation basics

Useful segment: pip install + requirements

PLAIN-ENGLISH EXPLANATION

Python docs explicitly treat venvs as disposable/non-portable. The durable contract is your source, dependency declaration and setup command. A recreated environment that passes the same tests is stronger evidence than a copied ``.venv`` directory.

C3-00-L05 - Follow with me once

TINY WORKED EXAMPLE

Create `.venv`, install `pytest`, save requirements, print interpreter path, delete `.venv`, recreate, reinstall from requirements and rerun tests.

FOLLOW WITH ME ONCE

1. Run `python3 --version`. If missing: `sudo apt update && sudo apt install -y python3 python3-venv python3-pip`.
2. Run `python3 -m venv .venv` then `source .venv/bin/activate`.
3. Verify `python -c "import sys; print(sys.executable)"` contains `.venv`.
4. Run `python -m pip install pytest`; create a tiny passing test; run `pytest -q`.
5. Record `python -m pip freeze > requirements.txt`, deactivate, delete only `.venv`, recreate and reinstall with `python -m pip install -r requirements.txt`; rerun tests.

COMMAND / STRUCTURE TO TYPE

```
python3 -m venv .venv
source .venv/bin/activate
python -c "import sys; print(sys.executable)"
python -m pip install pytest
printf 'def test_math():\n    assert 2 + 2 == 4\n' > test_smoke.py
pytest -q
python -m pip freeze > requirements.txt
deactivate
rm -rf .venv
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
pytest -q
```

EXPECTED RESULT

The second environment is newly created, the interpreter points inside `.venv`, and the same test passes after recreation. Git does not track `.venv`.

WHY IT WORKED

The proof tests the real dependency contract: the environment can disappear and the project can recover from saved instructions.

BREAK / FIX / DEBUG

Break it: `pytest` command not found after recreation. **Fix:** verify the active interpreter and install from requirements using `python -m pip`, not a random global `pip`.

COMMON BEGINNER MISTAKES

- Committing `.venv`.
- Copying a venv between machines and calling that reproducible.
- Using global `pip` while a different Python interpreter runs the project.
- Recording dependencies but never testing a clean recreation.

RECOVERY IF STUCK

Check `which python`, `python --version`, `python -m pip --version`, and `python -c "import sys; print(sys.executable)"`. If state is confused, preserve source/requirements, recreate only the disposable venv.

C3-00-L05 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

Create a second minimal project from scratch with one dependency/test. Recreate it only from README + dependency file and prove the test passes.

VALIDATION - DO NOT SKIP

- Fresh recreation succeeds.
- Active interpreter lives in project ``.venv``.
- Dependency instructions are sufficient.
- ``.venv`` is ignored by Git.

PRODUCTION / ARCHITECTURE REASONING

Later Airflow/Spark/project environments need explicit dependency boundaries. Environment recreation is a deployment and incident-recovery skill, not just setup hygiene.

TEACH IT BACK

In 60-90 seconds explain python virtual environments and dependency pinning in simple English, including one failure mode and how you validate correctness.

PROTECTED REVIEW + CHANGED RETRY

1) Save genuine P05 attempt in an append-only attempt folder. 2) Only then open the matching P05 section of C3-012. 3) Record the mistake/reason, close the review, and do the changed retry without copying the example. 4) If still weak, use targeted remediation; do not restart the whole phase.

EVIDENCE / MASTERY

Save version/interpreter output, requirements, first and recreated test output, Git ignore proof, changed transfer and explain-back.

Mastered only when: the changed retry passes its validation, you can explain the concept without the lesson/review open, and the evidence bundle is saved. A command merely running is not mastery.

C3-00-L06 - Python modules, packages and project structure

Focused effort: 2.5 h | Matching practice: P06 | Review stays protected until first attempt is saved.

WHY IT MATTERS

A 400-line script with reading, validation, business logic, CLI and output glued together is hard to test and reuse. Structure makes responsibility visible.

WHAT YOU LEARN

Separate reusable core logic from input/output and CLI wiring; understand modules, packages, imports and a simple `src/` project layout.

PREREQUISITE

C2B Python automation plus L05 environment discipline.

PICTURE IT IN YOUR HEAD

A giant script is a mixed drawer full of every tool. A package is a labeled toolbox: `core.py` does business logic, `io.py` handles files, `cli.py` translates command-line input, and tests can inspect pieces independently.

NEW WORDS BEFORE WE USE THEM

Word	Meaning in this lesson
module	A Python file that can define reusable functions/classes.
package	A namespace grouping related modules.
import	Load names from another module/package.
entry point	Boundary where a user/program starts the application.
pure-ish function	Logic whose output mostly depends on explicit inputs, making it easier to test.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [Python - Modules and packages](#)

Purpose: Split reusable logic into modules/packages

Useful segment: Modules + packages

PLAIN-ENGLISH EXPLANATION

Good structure does not mean dozens of files. It means boundaries match responsibilities. Keep core transformations callable without parsing command-line arguments. Keep CLI thin so tests can exercise logic directly.

C3-00-L06 - Follow with me once

TINY WORKED EXAMPLE

Refactor ``giant_script.py`` into ``src/filecheck/core.py``, ``src/filecheck/cli.py``, tests, and README. ``core.inspect_rows()`` accepts data; CLI parses paths/options then calls core.

FOLLOW WITH ME ONCE

1. Open P06 ``giant_script.py`` and mark four responsibilities: argument parsing, reading, validation/transformation, writing.
2. Create ``src/filecheck/`` with ``__init__.py``, ``core.py``, ``cli.py``.
3. Move reusable logic to ``core.py``; keep file/CLI boundary thin.
4. Run a direct import smoke check: ``python -c "from filecheck.core import ..."`` using the project setup instructions.
5. Write at least one test of core without invoking the CLI.

COMMAND / STRUCTURE TO TYPE

```
# mental structure
project/
  pyproject.toml
  src/filecheck/__init__.py
  src/filecheck/core.py      # reusable logic
  src/filecheck/cli.py      # argparse + exit contract
  tests/test_core.py
  README.md
```

EXPECTED RESULT

Core logic can run in a test without command-line parsing. CLI can call the same core. No business rule is duplicated between CLI and tests.

WHY IT WORKED

Each responsibility has one home. That reduces accidental coupling and gives unit tests a stable target.

BREAK / FIX / DEBUG

Break it: ``import `cli.py`` from a test and argument parsing immediately exits. **Fix:** move parser execution under a function/main boundary and move reusable logic to ``core.py``.

COMMON BEGINNER MISTAKES

- Creating many files with no responsibility boundary.
- Running CLI parsing at import time.
- Duplicating business logic in CLI and tests.
- Using relative filesystem assumptions inside core logic.

RECOVERY IF STUCK

Draw boxes for INPUT/CLI, CORE, OUTPUT. Move one responsibility at a time, run tests after each move, and inspect imports before adding more structure.



C3-00-L06 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

Refactor a different `daily_summary.py` utility so core aggregation is callable independently and the CLI is thin.

VALIDATION - DO NOT SKIP

- Core imports without side effects.
- Unit test reaches core without CLI/file-system ceremony where possible.
- CLI delegates rather than duplicates rules.
- Project layout is documented.

PRODUCTION / ARCHITECTURE REASONING

Packages become reusable components inside orchestration tasks, containers and CI pipelines. Clean boundaries reduce change blast radius.

TEACH IT BACK

In 60-90 seconds explain python modules, packages and project structure in simple English, including one failure mode and how you validate correctness.

PROTECTED REVIEW + CHANGED RETRY

1) Save genuine P06 attempt in an append-only attempt folder. 2) Only then open the matching P06 section of C3-012. 3) Record the mistake/reason, close the review, and do the changed retry without copying the example. 4) If still weak, use targeted remediation; do not restart the whole phase.

EVIDENCE / MASTERY

Save before/after structure, import smoke check, test output, changed refactor and 60-90 second explanation of module/package/core/CLI.

Mastered only when: the changed retry passes its validation, you can explain the concept without the lesson/review open, and the evidence bundle is saved. A command merely running is not mastery.

C3-00-L07 - Configuration, environment variables and secret boundaries

Focused effort: 2.0 h | Matching practice: P07 | Review stays protected until first attempt is saved.

WHY IT MATTERS

Hard-coded paths and credentials make software brittle and dangerous. Configuration should change behavior without editing source; secrets require a stricter boundary.

WHAT YOU LEARN

Apply a documented precedence rule, read non-secret configuration from environment variables/CLI, keep secrets out of source/logs/Git, and fail clearly on invalid config.

PREREQUISITE

L05-L06.

PICTURE IT IN YOUR HEAD

Source code is the machine. Configuration is the control panel. A secret is the key to the building - it may be needed by the machine but should not be painted on the control panel or printed in the logs.

NEW WORDS BEFORE WE USE THEM

Word	Meaning in this lesson
configuration	Runtime setting such as path, log level or batch size.
environment variable	Name/value supplied by the process environment.
secret	Credential/token/key whose disclosure can grant access.
precedence	Rule deciding which source wins when multiple settings exist.
default	Safe value used only when no higher-priority setting is supplied.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [Python 3.14 - Command line and environment](#)

Purpose: Environment-variable boundary and precedence mental model

Useful segment: Environment variables / command line behavior

PLAIN-ENGLISH EXPLANATION

For this course use a simple rule when relevant: CLI > environment variable > safe default. This is a course contract, not a universal law. Secrets are never committed, printed in evidence, or placed in example files except obvious fake placeholders.



C3-00-L07 - Follow with me once

TINY WORKED EXAMPLE

Output directory defaults to `./out`, environment `C3_OUTPUT_DIR` can override it, and CLI `--output-dir` wins over both. A fake `.env.example` contains `API_TOKEN=replace_me` only; real `.env` is ignored.

FOLLOW WITH ME ONCE

1. Add `.env` to `.gitignore`; create `.env.example` with fake placeholders only.
2. Implement a small `resolve_output_dir(cli_value, env)` function.
3. Set `export C3_OUTPUT_DIR=out_from_env`, run without CLI and verify that path wins over default.
4. Run with `--output-dir out_from_cli` and verify CLI wins.
5. Search tracked files/logs for the fake token pattern and demonstrate the secret boundary.

COMMAND / STRUCTURE TO TYPE

```
# precedence
CLI --output-dir value
    | if absent
    v
C3_OUTPUT_DIR environment variable
    | if absent
    v
./out safe default
```

EXPECTED RESULT

Three runs prove default, environment override, and CLI override. No real credential appears in source, Git history or logs.

WHY IT WORKED

The behavior is explicit and testable. Operational changes do not require source edits, while secrets stay outside durable/shared artifacts.

BREAK / FIX / DEBUG

Break it: config contains `BATCH_SIZE=abc` and the program crashes deep in processing. **Fix:** parse/validate config at startup and fail with a clear message before data work begins.

COMMON BEGINNER MISTAKES

- Hard-coding Windows paths inside reusable code.
- Logging an API token while debugging.
- Treating `.env.example` as a place for real credentials.
- Having several config sources with undocumented precedence.

RECOVERY IF STUCK

Print only non-sensitive resolved configuration, one field at a time. Confirm precedence with three tiny runs before debugging downstream code.

C3-00-L07 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

Add `C3_LOG_LEVEL`` with CLI override `--log-level``. Reject unsupported values early and prove precedence on a changed run.

VALIDATION - DO NOT SKIP

- Precedence is documented and demonstrated.
- Invalid values fail before processing.
- No secret is tracked/logged.
- Default is safe and predictable.

PRODUCTION / ARCHITECTURE REASONING

Cloud identities and secret managers come later, but the boundary starts here: code does not own secrets, logs do not reveal them, and config is validated at startup.

TEACH IT BACK

In 60-90 seconds explain configuration, environment variables and secret boundaries in simple English, including one failure mode and how you validate correctness.

PROTECTED REVIEW + CHANGED RETRY

1) Save genuine P07 attempt in an append-only attempt folder. 2) Only then open the matching P07 section of C3-012. 3) Record the mistake/reason, close the review, and do the changed retry without copying the example. 4) If still weak, use targeted remediation; do not restart the whole phase.

EVIDENCE / MASTERY

Save three config-resolution runs, ignore/search proof, invalid-config failure, changed transfer and explain-back.

Mastered only when: the changed retry passes its validation, you can explain the concept without the lesson/review open, and the evidence bundle is saved. A command merely running is not mastery.

C3-00-L08 - Structured logging and exception handling

Focused effort: 2.5 h | Matching practice: P08 | Review stays protected until first attempt is saved.

WHY IT MATTERS

When a pipeline fails at 2 AM, "something went wrong" is useless. Logs should tell the operator what ran, what failed, where, and what to do next - without exposing secrets.

WHAT YOU LEARN

Use logger names/levels, preserve exception context, distinguish expected validation failures from unexpected exceptions, and produce actionable evidence.

PREREQUISITE

C2B logging/error-handling basics plus L06-L07 boundaries.

PICTURE IT IN YOUR HEAD

Logs are an aircraft flight recorder, not a diary. Record events an operator can act on: run identity, stage, counts, validated assumptions, failure context. Do not dump every variable just because you can.

NEW WORDS BEFORE WE USE THEM

Word	Meaning in this lesson
log level	Severity category such as DEBUG/INFO/WARNING/ERROR.
logger	Named source of log events, often per module.
exception	Python signal that normal execution cannot continue.
stack trace	Call history showing where an unexpected exception traveled.
context	Safe metadata that makes an event diagnosable, e.g. file name/run id.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [Python - Logging HOWTO](#)

Purpose: Structured levels, logger names and exception evidence

Useful segment: Basic logging + advanced logging flow

RECOMMENDED REFERENCE: [Python - Errors and Exceptions](#)

Purpose: Catch only where recovery/context is meaningful

Useful segment: Exceptions + handling

PLAIN-ENGLISH EXPLANATION

Use `logging.getLogger(__name__)` inside modules. Catch exceptions only where you can add context, recover, translate to a stable failure contract, or guarantee cleanup. Do not swallow unexpected exceptions and return success.

C3-00-L08 - Follow with me once

TINY WORKED EXAMPLE

A reader logs INFO with safe file path/name, WARNING for recoverable skipped records if policy allows, ERROR for contract failure, and `logger.exception(...)` at the application boundary for an unexpected crash.

FOLLOW WITH ME ONCE

1. Configure logging once at the application boundary.
2. Add module logger via `logging.getLogger(\_\_name\_\_)`.
3. Log run start, input name, row count after validation, output path, and run completion.
4. Trigger a missing-file failure and an unexpected parsing error; compare evidence.
5. Confirm no token/secret values appear in logs.

COMMAND / STRUCTURE TO TYPE

```
import logging
logger = logging.getLogger(__name__)

def load_rows(path):
    logger.info('reading input path=%s', path)
    try:
        ...
    except FileNotFoundError:
        logger.error('input missing path=%s', path)
        raise
```

EXPECTED RESULT

A failed run is obviously failed, has enough safe context to diagnose, and does not publish a success message. A successful run leaves concise INFO evidence.

WHY IT WORKED

The lower layer adds domain/file context and preserves the exception. The application boundary decides the user-facing exit behavior, so failure is not silently converted to success.

BREAK / FIX / DEBUG

Break it: `except Exception: print("failed")` then exit 0. **Fix:** catch only known cases locally; preserve/raise unexpected errors; application boundary logs exception context and exits non-zero.

COMMON BEGINNER MISTAKES

- Logging every row and creating noise/PII risk.
- Catching `Exception` everywhere.
- Printing a secret while debugging.
- Writing ERROR but still returning success.

RECOVERY IF STUCK

Reproduce with the smallest input, increase log detail deliberately, inspect the first meaningful error, and follow the exception chain rather than adding random try/except blocks.



C3-00-L08 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

On a changed CSV utility, design INFO/WARNING/ERROR events and demonstrate one expected validation failure and one unexpected exception path.

VALIDATION - DO NOT SKIP

- Success/failure exit behavior matches logs.
- Errors include safe actionable context.
- Unexpected exceptions keep trace evidence at boundary.
- No secret/PII dump.

PRODUCTION / ARCHITECTURE REASONING

Observability later expands to metrics/traces/alerts, but trustworthy logs are the first operational record and incident timeline.

TEACH IT BACK

In 60-90 seconds explain structured logging and exception handling in simple English, including one failure mode and how you validate correctness.

PROTECTED REVIEW + CHANGED RETRY

1) Save genuine P08 attempt in an append-only attempt folder. 2) Only then open the matching P08 section of C3-012. 3) Record the mistake/reason, close the review, and do the changed retry without copying the example. 4) If still weak, use targeted remediation; do not restart the whole phase.

EVIDENCE / MASTERY

Save success and failure logs, exit codes, secret scan, changed transfer, and explain why "logged ERROR" is not enough if the process exits 0.

Mastered only when: the changed retry passes its validation, you can explain the concept without the lesson/review open, and the evidence bundle is saved. A command merely running is not mastery.

C3-00-L09 - Unit tests and integration-test boundaries

Focused effort: 3.0 h | Matching practice: P09 | Review stays protected until first attempt is saved.

WHY IT MATTERS

Manual checking proves only what you happened to look at. Tests turn critical behavior into repeatable evidence and protect refactors from quietly changing rules.

WHAT YOU LEARN

Write focused unit tests, use `tmp_path` for isolated filesystem behavior, distinguish unit vs integration scope, and recognize weak tests that only copy implementation.

PREREQUISITE

L05-L08. `pytest` should already exist in the project venv; install now if missing.

PICTURE IT IN YOUR HEAD

Testing a light switch on a workbench is a unit test; testing the switch wired into the room circuit is integration testing. Both matter, but they answer different questions.

NEW WORDS BEFORE WE USE THEM

Word	Meaning in this lesson
unit test	Checks a small unit of logic with controlled dependencies.
integration test	Checks that multiple real components work together at a boundary.
fixture	Prepared test resource/state.
assertion	Statement that expected behavior must be true.
regression	Previously working behavior breaks after change.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [pytest - API reference](#)

Purpose: pytest basics and `tmp_path` fixture

Useful segment: `tmp_path` + assertion behavior

RECOMMENDED VIDEO: [Tech With Tim - Pytest Tutorial](#)

Purpose: See first test, assertions, fixtures and test isolation visually

Useful segment: 00:00-07:13; 12:47-16:28

PLAIN-ENGLISH EXPLANATION

Prefer small deterministic unit tests for core rules and fewer integration tests for real boundaries such as filesystem/CLI/database later. A test that reproduces the same bug in its expected calculation is false confidence, so include independent reasoning and edge cases.

C3-00-L09 - Follow with me once

TINY WORKED EXAMPLE

Unit: ``required_missing(columns, required)`` returns missing names. Filesystem integration-ish test: create an isolated CSV in `pytest `tmp_path``, run your reader, verify output artifact and error behavior.

FOLLOW WITH ME ONCE

1. Run ``pytest --version``; if missing while `venv` active, ``python -m pip install pytest``.
2. Write one passing unit test and one deliberate failing assertion; read the failure output.
3. Use ``tmp_path`` to create a temporary input without touching your real course data.
4. Add tests for happy path, missing column, empty file, and duplicate rerun behavior.
5. Run ``pytest -q``; then deliberately break core logic and prove a test fails.

COMMAND / STRUCTURE TO TYPE

```
def test_missing_required():
    from filecheck.core import missing_required
    assert missing_required(['id', 'name'], {'id', 'email'}) == {'email'}

def test_tmp_file(tmp_path):
    p = tmp_path / 'people.csv'
    p.write_text('id,name\n1,Ada\n', encoding='utf-8')
    assert p.exists()
```

EXPECTED RESULT

The suite passes when behavior is correct and fails for your deliberate regression. Tests leave no uncontrolled files in the project root.

WHY IT WORKED

The unit test checks a business/validation rule directly; ``tmp_path`` isolates filesystem behavior so tests are repeatable and do not pollute real data.

BREAK / FIX / DEBUG

Break it: change validator to always return no missing columns. A good negative test fails immediately. If all tests still pass, the suite is not protecting that contract.

COMMON BEGINNER MISTAKES

- Only happy-path tests.
- Tests that depend on execution order or a real desktop file.
- Asserting the same formula used by implementation without independent expected values.
- Calling every test an integration test.

RECOVERY IF STUCK

Start from one input and one expected output. Make the function smaller if setup is huge. Use ``tmp_path``; inspect the first failure; do not weaken assertions just to get green.



C3-00-L09 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

For a changed file-inspector function, write at least 4 tests including one edge case and one failure contract. Deliberately inject one regression and prove the suite catches it.

VALIDATION - DO NOT SKIP

- Deliberate regression causes red.
- Clean code causes green.
- Filesystem tests are isolated.
- Test names communicate behavior.

PRODUCTION / ARCHITECTURE REASONING

CI later runs these tests automatically on every change. The skill here is not pytest syntax; it is deciding which contracts deserve automated protection.

TEACH IT BACK

In 60-90 seconds explain unit tests and integration-test boundaries in simple English, including one failure mode and how you validate correctness.

PROTECTED REVIEW + CHANGED RETRY

1) Save genuine P09 attempt in an append-only attempt folder. 2) Only then open the matching P09 section of C3-012. 3) Record the mistake/reason, close the review, and do the changed retry without copying the example. 4) If still weak, use targeted remediation; do not restart the whole phase.

EVIDENCE / MASTERY

Save test file, red-before-green proof for one deliberate regression, final `pytest -q``, changed transfer, and unit-vs-integration explain-back.

Mastered only when: the changed retry passes its validation, you can explain the concept without the lesson/review open, and the evidence bundle is saved. A command merely running is not mastery.

C3-00-L10 - CLI design and reusable data utilities

Focused effort: 2.5 h | Matching practice: P10 | Review stays protected until first attempt is saved.

WHY IT MATTERS

A stable CLI lets humans, schedulers and containers invoke the same utility predictably. If core logic is tied to manual editing or notebook state, orchestration becomes fragile.

WHAT YOU LEARN

Build a thin `argparse` CLI, provide useful `--help`, define exit codes/output contract, and keep core logic reusable outside CLI parsing.

PREREQUISITE

L06-L09.

PICTURE IT IN YOUR HEAD

The CLI is a front desk. It accepts a request, checks the form, translates it into clean arguments for the back-office core, then reports success/failure. The front desk should not secretly do the whole job.

NEW WORDS BEFORE WE USE THEM

Word	Meaning in this lesson
CLI	Command-line interface.
argument	Value supplied when launching a command.
flag/option	Named CLI setting such as <code>--output</code> .
<code>--help</code>	Self-documenting usage contract.
exit code	Machine-readable process result.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [Python - argparse](#)

Purpose: Stable beginner CLI contract and `--help`

Useful segment: Parser basics + subcommands only if needed

PLAIN-ENGLISH EXPLANATION

Use `argparse` for a basic Python CLI. Parse arguments in the CLI layer, validate CLI-specific constraints, call reusable core, write predictable output, and return stable exit behavior. Keep business logic testable without launching a subprocess.

C3-00-L10 - Follow with me once

TINY WORKED EXAMPLE

``python -m filecheck.cli data.csv --output out/profile.json --log-level INFO``. Exit 0 success; exit 2 invalid usage/contract; non-zero unexpected failure. ``--help`` explains inputs/options.

FOLLOW WITH ME ONCE

1. Build parser with positional input path plus ``--output`` and ``--log-level``.
2. Run ``python -m filecheck.cli --help`` before implementing full work; make usage understandable.
3. Call core function with parsed values.
4. Return/translate known validation failure to a documented non-zero code.
5. Test core directly; add at least one CLI-level test for help or exit behavior.

COMMAND / STRUCTURE TO TYPE

```
python -m filecheck.cli --help
python -m filecheck.cli data/valid_people.csv --output out/profile.json
echo $?
# now invalid
python -m filecheck.cli data/missing.csv --output out/profile.json
echo $?
```

EXPECTED RESULT

A new learner can run ``--help``, understand the command, invoke success, and distinguish invalid input from successful processing. Core remains callable separately.

WHY IT WORKED

The CLI is a stable process boundary. Humans and later orchestrators use the same command while core stays reusable/testable.

BREAK / FIX / DEBUG

Break it: CLI writes an error message but still exits 0. **Fix:** translate known failure into non-zero status and test that behavior.

COMMON BEGINNER MISTAKES

- Business logic inside ``if __name__ == "__main__":`` only.
- No ``--help`` or ambiguous argument names.
- Changing output path/name every run.
- Returning success after validation failure.

RECOVERY IF STUCK

Run ``--help``, test one valid invocation, then one intentionally invalid invocation. If behavior is unclear, simplify parser and document exit contract before adding features.

C3-00-L10 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

Build a changed `rowcount` utility CLI with input path and JSON output, one validation error, stable help and a reusable core function.

VALIDATION - DO NOT SKIP

- Help is understandable.
- Success and failure exit codes differ.
- Output schema/path are stable.
- Core can be tested without CLI parsing.

PRODUCTION / ARCHITECTURE REASONING

Schedulers such as Airflow and containers depend on stable commands, exit codes and outputs. A good CLI is a clean automation seam.

TEACH IT BACK

In 60-90 seconds explain cli design and reusable data utilities in simple English, including one failure mode and how you validate correctness.

PROTECTED REVIEW + CHANGED RETRY

1) Save genuine P10 attempt in an append-only attempt folder. 2) Only then open the matching P10 section of C3-012. 3) Record the mistake/reason, close the review, and do the changed retry without copying the example. 4) If still weak, use targeted remediation; do not restart the whole phase.

EVIDENCE / MASTERY

Save help output, success/failure commands + exit codes, output artifact, core test, changed transfer and explain-back.

Mastered only when: the changed retry passes its validation, you can explain the concept without the lesson/review open, and the evidence bundle is saved. A command merely running is not mastery.

C3-00-L11 - Docker and container mental model

Focused effort: 3.0 h | Matching practice: P11 | Review stays protected until first attempt is saved.

WHY IT MATTERS

Containers package an application environment so the same defined image can run consistently across machines. But a container does not automatically make bad code reproducible or secure.

WHAT YOU LEARN

Distinguish image vs container, install/verify Docker Desktop only at first use, run a tiny container, read a minimal Dockerfile, understand mounts/ephemeral state, and document an honest fallback if Docker cannot run.

PREREQUISITE

L05-L10. Docker is intentionally installed only now if absent. This lesson can use an honest design-only fallback on a machine where Docker Desktop cannot safely run.

PICTURE IT IN YOUR HEAD

An image is a sealed recipe + ingredients + kitchen setup packed for delivery. A container is one running kitchen opened from that package. You can start several containers from the same image; changing a running container is not the same as changing the image recipe.

NEW WORDS BEFORE WE USE THEM

Word	Meaning in this lesson
image	Read-only packaged filesystem/config used to start containers.
container	Running instance of an image.
Dockerfile	Text instructions used to build an image.
layer	Reusable image filesystem step.
mount	Host/storage path made available inside a container.
ephemeral	Expected to disappear unless state is stored deliberately outside the container.

USEFUL VIDEO / SOURCE

RECOMMENDED REFERENCE: [Docker - Get started](#)

Purpose: Image/container mental model

Useful segment: Image vs container + first run

RECOMMENDED REFERENCE: [Docker Desktop for Windows](#)

Purpose: Install only at first use if Docker is absent

Useful segment: WSL2 backend install + verify

RECOMMENDED VIDEO: [Docker - Intro and Overview on-demand training](#)

Purpose: Visualize image/container lifecycle and basic containerization

Useful segment: 01:34-10:20; optional 29:57-32:59

PLAIN-ENGLISH EXPLANATION

Check Docker first; do not install it earlier. On Windows, Docker Desktop commonly uses the WSL2 backend. Verification requires both client and server/engine information. A real run proves more than a screenshot of installation.



C3-00-L11 - Follow with me once

TINY WORKED EXAMPLE

Containerize a tiny Python CLI with a minimal Dockerfile. Run `--help`, then run against a mounted input/output directory so evidence persists outside the container.

FOLLOW WITH ME ONCE

1. Run `docker version`. If client/server are available, do not reinstall.
2. If missing, install Docker Desktop for Windows following current official instructions, keep WSL2 backend, restart if required, then reopen WSL.
3. Verify `docker version` shows both Client and Server.
4. Run `docker run --rm hello-world` or another approved tiny image.
5. Build the course minimal image and run its `--help`. Then mount an input/output directory and verify output persists after container exits.

COMMAND / STRUCTURE TO TYPE

```
# verify
docker version
docker run --rm hello-world

# mental Dockerfile
FROM python:3.14-slim
WORKDIR /app
COPY . /app
RUN python -m pip install --no-cache-dir -r requirements.txt
ENTRYPOINT ["python", "-m", "filecheck.cli"]

# after build
docker run --rm filecheck:local --help
```

EXPECTED RESULT

Real path: `docker version` shows Client + Server and a tiny container exits successfully. **Fallback path:** evidence explicitly says Docker was unavailable and completes only the design/command reasoning - never fabricated runtime success.

WHY IT WORKED

The image defines runtime dependencies and entry point; the container is disposable; mounted evidence survives. Verification distinguishes "installed client" from a working engine.

BREAK / FIX / DEBUG

Break it: `docker version` prints Client but cannot connect to daemon. **Fix:** do not reinstall Python/Git; verify Docker Desktop/WSL integration and engine state. Record the exact error if hardware/policy blocks use.

COMMON BEGINNER MISTAKES

- Installing Docker at C3-00 start even though not needed yet.
- Calling an image a running container.
- Writing output only inside an ephemeral container and losing evidence.
- Claiming Docker success without a real command run.

RECOVERY IF STUCK

Check `docker version`, Docker Desktop engine state, WSL integration, then a tiny known image. Separate installation/engine/network/build problems rather than changing everything at once.



C3-00-L11 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

Explain and, if available, run a changed containerized `rowcount` CLI with a host-mounted output. If Docker cannot safely run, produce the Dockerfile + exact commands + expected result + blocker evidence.`

VALIDATION - DO NOT SKIP

- Image vs container explanation correct.
- Client/server engine verified or blocker recorded honestly.
- `--help` works in real container when available.`
- Output persistence/mount reasoning is correct.

PRODUCTION / ARCHITECTURE REASONING

Later Airflow/local stacks and CI jobs rely on container boundaries. You still need tests, configs, secrets and observability; Docker does not replace those controls.

TEACH IT BACK

In 60-90 seconds explain image vs container, why a working Docker client is not proof the engine works, and what evidence you save if Docker cannot run.

PROTECTED REVIEW + CHANGED RETRY

1) Save genuine P11 attempt in an append-only attempt folder. 2) Only then open the matching P11 section of C3-012. 3) Record the mistake/reason, close the review, and do the changed retry without copying the example. 4) If still weak, use targeted remediation; do not restart the whole phase.

EVIDENCE / MASTERY

Save Docker version, real tiny run/build/run evidence OR explicit fallback blocker, Dockerfile, mount explanation, changed transfer and explain-back.

Mastered only when: the changed retry passes its validation, you can explain the concept without the lesson/review open, and the evidence bundle is saved. A command merely running is not mastery.

C3-00-L12 - Idempotence, reproducibility and runbooks

Focused effort: 3.0 h | Matching practice: P12 | Review stays protected until first attempt is saved.

WHY IT MATTERS

Production failures cause reruns. If rerunning doubles rows, overwrites good data unpredictably or requires the original developer to remember magic steps, the system is fragile.

WHAT YOU LEARN

Design safe reruns, distinguish deterministic result from identical bytes, create a runbook, prove repeated execution behavior, and prepare operator handoff.

PREREQUISITE

L01-L11.

PICTURE IT IN YOUR HEAD

Pressing an elevator button twice should not summon two elevators for the same request. A safe data rerun similarly reaches the intended state without multiplying the business effect.

NEW WORDS BEFORE WE USE THEM

Word	Meaning in this lesson
idempotence	Repeating an operation leads to the same intended state/effect.
determinism	Same controlled inputs produce the same defined result.
reproducibility	Another clean environment/operator can recreate the result from instructions.
checkpoint	Recorded progress/state used to resume safely.
runbook	Operator instructions for setup, run, validate, failure, recovery, rerun and escalation.

USEFUL VIDEO / SOURCE

REQUIRED REFERENCE: Course-built rerun/runbook standard

Purpose: Idempotence, deterministic evidence and operator handoff

Useful segment: Entire lesson; no external dependency

PLAIN-ENGLISH EXPLANATION

Idempotence is about intended effect, not necessarily byte-for-byte identical logs/timestamps. Reproducibility includes environment, config, inputs, commands and validation. A runbook should be short enough to use during failure but complete enough that another person does not need hidden knowledge.



C3-00-L12 - Follow with me once

TINY WORKED EXAMPLE

A profile utility writes `out/profile.json` atomically from a fixed input. Two unchanged runs produce the same business fields/hash for deterministic content; logs may have different timestamps. A runbook states how to verify input, run, validate, recover and rerun.

FOLLOW WITH ME ONCE

1. Run your C3-00 CLI twice on unchanged input/config.
2. Compare business output; use `sha256sum` if byte-identical output is part of your contract.
3. Create one failed run before publish and prove the prior known-good output remains intact.
4. Write `RUNBOOK.md`: prerequisites, verify environment, command, expected result, validation, common failure, recovery, safe rerun, evidence, owner/escalation.
5. Give the runbook to your future-self test: could you operate after forgetting the implementation details?

COMMAND / STRUCTURE TO TYPE

```
python -m filecheck.cli data/valid_people.csv --output out/profile.json
sha256sum out/profile.json > evidence/run1.sha256
python -m filecheck.cli data/valid_people.csv --output out/profile.json
sha256sum out/profile.json > evidence/run2.sha256
diff evidence/run1.sha256 evidence/run2.sha256
# If timestamps are part of output, compare defined business fields instead of
demanding identical bytes.
```

EXPECTED RESULT

Two unchanged runs do not duplicate business effect. The runbook contains exact commands and checks. A deliberate pre-publish failure does not destroy the prior valid output.

WHY IT WORKED

The proof checks rerun behavior against an explicit contract. The runbook then transfers that operational knowledge to another operator.

BREAK / FIX / DEBUG

Break it: use append mode on rerun and row count doubles. Fix: choose a deliberate publish/replay strategy, validate before publish, and make the rerun rule part of the contract/tests.

COMMON BEGINNER MISTAKES

- Calling every repeated command idempotent without checking business effect.
- Demanding identical logs when timestamps legitimately differ.
- A runbook that says only "run script.py".
- Recovery instructions that delete state before diagnosis.

RECOVERY IF STUCK

Restate the intended end state. Compare input identity/config, inspect previous output, run on a copy/test path, then choose replay/overwrite behavior deliberately.

C3-00-L12 - Independent transfer & mastery

INDEPENDENT SMALL VERSION

Changed domain: daily inventory snapshot. Define safe rerun semantics, validate two unchanged reruns, create one failure/recovery drill and write a concise operator runbook.

VALIDATION - DO NOT SKIP

- Rerun rule matches the business/output grain.
- Repeated execution does not multiply intended effect.
- Failure recovery preserves known-good state where contract requires.
- Another operator can follow runbook without hidden steps.

PRODUCTION / ARCHITECTURE REASONING

This is the foundation for Airflow retries, Spark reprocessing, Kafka deduplication, Delta MERGE and incident runbooks. Idempotence will return repeatedly at larger scale.

TEACH IT BACK

In 60-90 seconds explain idempotence vs determinism vs reproducibility and one concrete rerun failure.

PROTECTED REVIEW + CHANGED RETRY

1) Save genuine P12 attempt in an append-only attempt folder. 2) Only then open the matching P12 section of C3-012. 3) Record the mistake/reason, close the review, and do the changed retry without copying the example. 4) If still weak, use targeted remediation; do not restart the whole phase.

EVIDENCE / MASTERY

Save run1/run2 comparison, one failure/recovery drill, RUNBOOK.md, changed inventory-snapshot transfer, and explain idempotence vs determinism vs reproducibility.

Mastered only when: the changed retry passes its validation, you can explain the concept without the lesson/review open, and the evidence bundle is saved. A command merely running is not mastery.



C3-00 exit checkpoint

After all 12 lessons are mastered, complete the Dataset Inspector Mini-Lab. Gate A stays locked until lesson mastery + Mini-Lab evidence are complete. Gate B/C are protected fresh full retests and are never automatically unlocked by a repair.

C3-00 does not certify job readiness. It establishes the operating habits required before files/APIs, ETL/ELT, orchestration, Spark, lakehouse, streaming, cloud and DataOps modules.

Mastery checklist + revision quick reference

- ☐ Every lesson: guided rebuild completed, genuine independent attempt saved, validation recorded.
- ☐ Protected review opened only after the attempt; changed retry solved after closing the review.
- ☐ Explain-back: business/engineering purpose, grain or state, one failure mode, one recovery step, one validation check.
- ☐ Mini-Lab / integrated drill completed with reproducible evidence.
- ☐ Gate A attempted independently. If NEEDS_REPAIR: review viewed, targeted repair completed, changed micro-proof recorded, then a fresh B/C assigned explicitly.
- ☐ No unresolved critical failure, secret/private data leakage, fabricated live-run evidence or copied Gate solution.
- ☐ Evidence is saved with enough context for another person to reproduce or inspect it.

Revision quick reference

Wrong result / wrong grain	Return to validation + data contract / grain lesson; reproduce smallest failing case.
Works once, rerun breaks	Return to idempotence/state/checkpoint/recovery lesson; prove clean replay.
Cannot explain failure	Return to mental model + break/fix section; write the failure boundary in your own words.
Gate NEEDS_REPAIR	Open only its protected review, repair named weak competency, solve changed micro-proof, then assign fresh retest.
Tool/vendor changed	Use the current official source for syntax/UI, but keep the book's durable engineering contract and validation.